

Processor Architecture

1. Overview

Processor architecture is the design of a CPU: what parts it contains, how it executes instructions, how it communicates with memory, and how hardware choices affect performance.

In simple terms, it answers:

- What does the CPU understand?
- How does it fetch and execute instructions?
- Where does it store temporary data?
- How does it control arithmetic, memory access, branching, and parallel execution?

Definition

Processor architecture is the organization and behavior of a processor, including its instruction set, registers, datapath, control unit, execution units, memory interface, and performance features such as pipelining, superscalar execution, and out-of-order execution.

Why It Matters

Processor architecture matters because every program eventually becomes machine instructions executed by hardware. Even high-level code in Java, Python, JavaScript, or C++ is ultimately translated or interpreted into operations the CPU can perform.

Understanding processor architecture helps you reason about:

- Why some operations are faster than others
- How loops, branches, and memory accesses affect performance
- How compilers generate machine code
- Why CPU caches and registers matter
- How operating systems switch between processes
- Why modern CPUs can execute multiple instructions at once

Where It Is Used in Real Systems

Processor architecture appears in:

- Operating systems: context switching, interrupts, system calls, scheduling
- Compilers: register allocation, instruction selection, optimization
- Databases: query execution, memory locality, vectorized processing
- Browsers: JavaScript engines, JIT compilation, sandboxing
- Backend servers: CPU-bound request handling, encryption, compression
- Embedded systems: low-power CPU design, real-time control
- Security: speculative execution attacks, privilege levels, memory protection

Why Interviewers Ask About It

Interviewers ask processor architecture questions to check whether you understand what happens below the programming language level.

They often want to see if you can explain:

- How an instruction is executed
- Difference between registers, memory, and cache
- RISC vs CISC
- Control unit vs datapath
- Hardwired vs microprogrammed control
- Addressing modes and instruction formats
- Why modern CPUs use pipelining, superscalar execution, and out-of-order execution

For placements, this topic is common in computer organization rounds, operating system rounds, embedded roles, compiler roles, and performance-focused SDE interviews.

2. Core Idea

The CPU is like a very fast worker that follows simple instructions one by one. Each instruction tells it to do something small: add two numbers, load a value from memory, store a result, compare values, or jump to another instruction.

The CPU repeatedly performs this cycle:

1. Fetch the next instruction from memory.
2. Decode what the instruction means.
3. Execute the operation.
4. Store the result.
5. Move to the next instruction.

This is called the fetch-decode-execute cycle.

Intuition

Think of the CPU as a kitchen worker following recipe steps:

Computer Term	Kitchen Analogy
Program	Recipe
Instruction	One recipe step
Program counter	Bookmark showing current recipe step
Instruction register	Step currently being read
Registers	Small bowls kept near the worker
Main memory	Pantry
ALU	Cutting/mixing tool
Control unit	Head chef deciding what happens next

Computer Term	Kitchen Analogy
Datapath	Physical movement of ingredients
Control path	Signals telling tools when to work

The worker is fastest when ingredients are already in nearby bowls. If every step requires walking to the pantry, execution slows down. Similarly, CPUs prefer registers and cache over main memory.

Small Example

High-level statement:

```
c = a + b;
```

Possible machine-level steps:

```
LOAD R1, [a]      ; copy value of a from memory into register R1
LOAD R2, [b]      ; copy value of b from memory into register R2
ADD  R3, R1, R2   ; R3 = R1 + R2
STORE [c], R3     ; copy result from R3 into memory location c
```

Even a simple line of code becomes several machine instructions.

Step-by-Step Execution

For instruction **ADD R3, R1, R2**:

1. Program counter contains the address of the **ADD** instruction.
2. CPU fetches the instruction from memory.
3. Instruction register stores the fetched instruction.
4. Control unit decodes it as an arithmetic operation.
5. Register file reads values from **R1** and **R2**.
6. ALU adds the values.
7. Result is written into **R3**.
8. Program counter moves to the next instruction.

The entire processor architecture exists to make this repeated process correct and fast.

3. Important Subtopics

3.1 CPU Components

What It Means

The CPU is made of several cooperating units:

Component	Purpose
ALU	Performs arithmetic and logical operations
Control unit	Directs execution by generating control signals
Registers	Very small, very fast storage inside CPU
Program counter	Holds address of next instruction
Instruction register	Holds current instruction
Datapath	Carries data between CPU units
Control path	Carries control signals
Cache interface	Connects CPU to faster memory close to processor
Bus interface	Connects CPU to memory and I/O devices
Clock	Synchronizes CPU operations

Why It Matters

Interviewers expect you to know that a CPU is not a single magic block. It is a collection of specialized parts. Performance depends on how well these parts cooperate.

Example

When executing `ADD R1, R2, R3`:

- Registers provide operands.
- ALU performs addition.
- Control unit selects the ALU operation.
- Datapath carries values.
- Register file stores the result.

Common Interview Angle

Question: "What are the main components of a CPU?"

Good answer: "A CPU mainly contains an ALU, control unit, registers, program counter, instruction register, datapath, and memory/bus interface. The ALU computes, the control unit coordinates, registers store temporary values, and the PC/IR manage instruction flow."

3.2 ALU, Control Unit, and Registers

What It Means

The ALU, control unit, and registers are the three core internal blocks of a CPU.

Unit	Role
ALU	Calculates results
Control unit	Decides what should happen
Registers	Hold operands, addresses, and results

ALU

The Arithmetic Logic Unit performs:

- Addition
- Subtraction
- Increment/decrement
- Bitwise AND, OR, XOR, NOT
- Shifts and rotates
- Comparisons

Example:

```
R3 = R1 + R2
```

The ALU receives values from registers **R1** and **R2**, adds them, and outputs the result.

Control Unit

The control unit decodes instructions and generates control signals.

For example, for **LOAD R1, [1000]**, it may generate signals like:

- Read memory
- Select memory address bus
- Enable register write
- Write result into **R1**

Registers

Registers are small storage locations inside the CPU.

Common types:

Register Type	Use
General-purpose registers	Hold temporary data
Program counter	Address of next instruction
Instruction register	Current instruction
Stack pointer	Top of current stack

Register Type	Use
Status/flags register	Zero, carry, sign, overflow flags
Memory address register	Address for memory operation
Memory data register	Data read from or written to memory

Why It Matters

Registers are much faster than memory. Compilers try to keep frequently used values in registers because memory access is expensive.

Example

Instead of repeatedly accessing memory:

```
LOAD R1, [x]
ADD R1, R1, 1
STORE [x], R1
```

The CPU works faster when `x` remains in a register across multiple operations.

Common Interview Angle

Question: "Why are registers faster than memory?"

Answer: "Registers are inside the CPU and directly connected to the datapath, so they can usually be accessed in one CPU cycle. Main memory is outside the CPU core and requires bus/cache access, which takes many more cycles."

3.3 Program Counter

What It Means

The program counter, or PC, is a CPU register that stores the address of the next instruction to fetch.

It is also called the instruction pointer in some architectures, such as `RIP` in x86-64.

Why It Matters

The PC controls program flow. Sequential execution, jumps, function calls, returns, loops, and branches all work by changing the PC.

Example

Assume instructions are stored like this:

Address	Instruction
1000	LOAD R1, [x]
1004	ADD R1, R1, 1
1008	STORE [x], R1
1012	HALT

Execution:

1. PC = 1000, fetch **LOAD**.
2. PC becomes 1004.
3. Fetch **ADD**.
4. PC becomes 1008.
5. Fetch **STORE**.
6. PC becomes 1012.

For a branch:

```
JMP 2000
```

The PC is updated to **2000** instead of the next sequential address.

Common Interview Angle

Question: "What happens to the program counter during a branch?"

Answer: "Normally the PC increments to the next instruction. During a branch or jump, the control unit loads the branch target address into the PC."

3.4 Instruction Register

What It Means

The instruction register, or IR, stores the instruction currently being decoded or executed.

Why It Matters

The CPU needs a stable copy of the current instruction while the control unit decodes it and generates control signals.

Example

If memory returns the binary instruction:

```
0001 0010 0011 0100
```

The CPU stores it in the instruction register. The control unit then interprets fields such as opcode, source register, destination register, and addressing mode.

Common Interview Angle

Question: "Difference between PC and IR?"

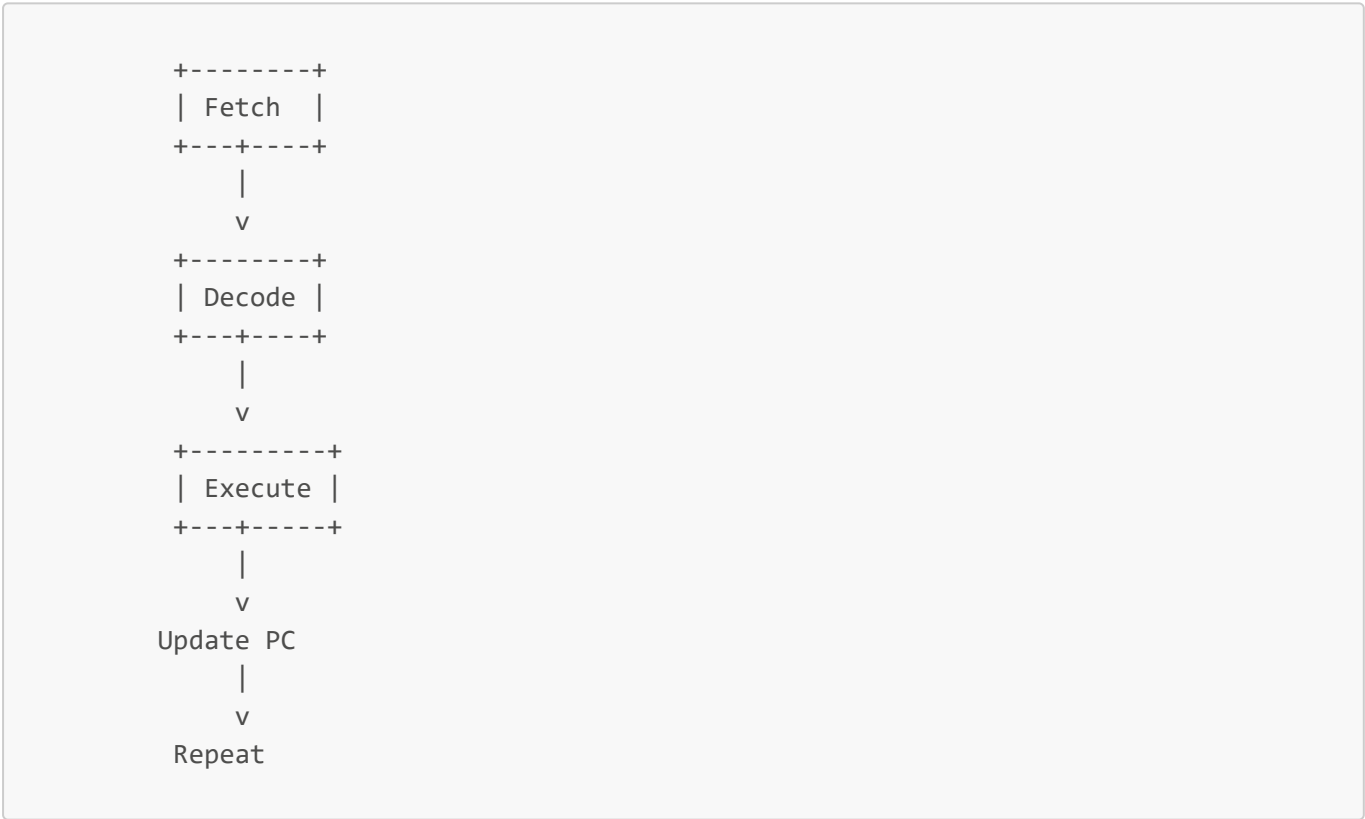
Register	Holds
Program counter	Address of next instruction
Instruction register	Current fetched instruction

Common mistake: Saying the PC stores the current instruction. It stores an address, not the instruction itself.

3.5 Fetch-Decode-Execute Cycle

What It Means

The fetch-decode-execute cycle is the basic process by which a CPU runs a program.



Steps

Step	What Happens
Fetch	CPU reads instruction from memory using PC
Decode	Control unit identifies operation and operands
Execute	ALU/memory/registers perform the operation

Step	What Happens
Write-back	Result is written to register or memory
PC update	PC moves to next instruction or branch target

Why It Matters

This cycle is the foundation for understanding pipelines, interrupts, branch prediction, and instruction-level parallelism.

Example

Instruction:

```
ADD R1, R2, R3
```

Flow:

1. Fetch instruction at PC.
2. Decode opcode as **ADD**.
3. Read **R2** and **R3**.
4. ALU adds values.
5. Write result into **R1**.
6. Increment PC.

Common Interview Angle

Question: "What is the fetch-decode-execute cycle?"

Answer: "It is the repeated CPU process of fetching the next instruction from memory, decoding its operation and operands, executing it using CPU units, writing back any result, and updating the program counter."

3.6 Instruction Set Architecture

What It Means

Instruction Set Architecture, or ISA, is the contract between software and hardware. It defines what instructions a processor can execute and how software sees the machine.

ISA specifies:

- Machine instructions
- Registers visible to programmers
- Data types
- Addressing modes
- Memory model
- Instruction formats

- Exception and interrupt behavior
- Privilege levels

Examples:

- x86-64
- ARM
- RISC-V
- MIPS
- PowerPC

Why It Matters

Software compiled for one ISA usually cannot run directly on a different ISA.

For example:

- A Windows x86-64 program cannot directly run on ARM unless recompiled or translated.
- Android phones commonly use ARM ISA.
- Many teaching processors use MIPS or RISC-V because they are cleaner to understand.

ISA vs Microarchitecture

Concept	Meaning
ISA	What the CPU promises to software
Microarchitecture	How a specific CPU implements that ISA

Example:

Two Intel CPUs may both support x86-64 ISA but have different cache sizes, pipelines, branch predictors, and execution units.

Common Interview Angle

Question: "What is an ISA?"

Answer: "ISA is the programmer-visible specification of a processor: its instructions, registers, addressing modes, data types, and behavior. It is the interface between software and hardware."

3.7 RISC vs CISC

What It Means

RISC and CISC are two design philosophies for instruction sets.

Term	Full Form	Basic Idea
RISC	Reduced Instruction Set Computer	Simple instructions, usually fixed length
CISC	Complex Instruction Set Computer	Rich instructions, often variable length

RISC

RISC processors use simpler instructions that typically execute in fewer cycles and are easier to pipeline.

Common RISC features:

- Fixed-length instructions
- Load-store architecture
- Many general-purpose registers
- Simple addressing modes
- Compiler does more work

Examples:

- ARM
- RISC-V
- MIPS

CISC

CISC processors provide more complex instructions that may do multiple low-level operations in one instruction.

Common CISC features:

- Variable-length instructions
- More addressing modes
- Memory operands allowed in many instructions
- More complex decoding
- Hardware does more work

Example:

- x86/x86-64

Why It Matters

RISC designs are usually easier to pipeline and power-efficient. CISC designs can provide compact machine code and backward compatibility.

Modern distinction is blurred. Modern x86 CPUs often decode complex x86 instructions into simpler internal micro-operations.

Example

CISC-style instruction:

```
ADD [memory_address], immediate_value
```

This may load from memory, add, and store back.

RISC-style sequence:

```
LOAD R1, [memory_address]
ADD R1, R1, immediate_value
STORE [memory_address], R1
```

Common Interview Angle

Question: "Is RISC always faster than CISC?"

Answer: "Not necessarily. RISC instructions are simpler and pipeline-friendly, but real performance depends on microarchitecture, clock speed, cache, compiler, workload, branch prediction, and power limits."

3.8 Register Architecture

What It Means

Register architecture describes how instructions access operands using registers, memory, stacks, or accumulators.

Main styles:

Architecture	Operand Source
Stack architecture	Operands are on stack
Accumulator architecture	One implicit accumulator register
Register-memory architecture	Operands may be registers or memory
Load-store architecture	ALU operations only use registers

Stack Architecture

Instructions operate on top of stack.

Example:

```
PUSH A
PUSH B
ADD
POP C
```

Used conceptually in virtual machines such as JVM bytecode, although actual hardware may differ.

Accumulator Architecture

One special register, the accumulator, is used implicitly.

Example:

```
LOAD A
ADD B
STORE C
```

Here **ADD B** means accumulator = accumulator + B.

Register-Memory Architecture

Instructions may directly use memory operands.

Example:

```
ADD R1, [x]
```

x86 supports this style.

Load-Store Architecture

Only load and store instructions access memory. Arithmetic instructions use registers only.

Example:

```
LOAD R1, [x]
LOAD R2, [y]
ADD R3, R1, R2
STORE [z], R3
```

ARM, RISC-V, and MIPS use this style.

Why It Matters

Register architecture affects instruction size, compiler design, performance, and pipeline complexity.

Common Interview Angle

Question: "Why do RISC processors usually use load-store architecture?"

Answer: "Because separating memory access from arithmetic keeps instructions simple, regular, and easier to pipeline."

3.9 Addressing Modes

What It Means

Addressing modes define how an instruction finds its operands.

An operand may be:

- Inside the instruction
- In a register
- In memory at a direct address
- In memory at an address computed from registers and offsets

Common Addressing Modes

Addressing Mode	Meaning	Example
Immediate	Operand value is inside instruction	<code>MOV R1, #5</code>
Register	Operand is in register	<code>ADD R1, R2</code>
Direct	Instruction contains memory address	<code>LOAD R1, [1000]</code>
Indirect	Register/memory contains address	<code>LOAD R1, [R2]</code>
Base + offset	Address = base register + constant	<code>LOAD R1, [R2 + 8]</code>
Indexed	Address = base + index	<code>LOAD R1, [ARR + R2]</code>
Relative	Address = PC + offset	<code>BEQ label</code>
Implied	Operand is implicit	<code>CLR CARRY</code>

Why It Matters

Addressing modes are heavily used in:

- Arrays
- Pointers
- Structures/classes
- Stack frames
- Branches
- Function calls

Example

C code:

```
x = arr[i];
```

Possible addressing:

```
LOAD R1, [ARR_BASE + i * 4]
```

For a 4-byte integer array, the address is base address plus index times element size.

Common Interview Angle

Question: "What is immediate addressing?"

Answer: "Immediate addressing means the operand value is part of the instruction itself, such as `MOV R1, #10`, where 10 is not fetched from memory as data."

3.10 Instruction Formats

What It Means

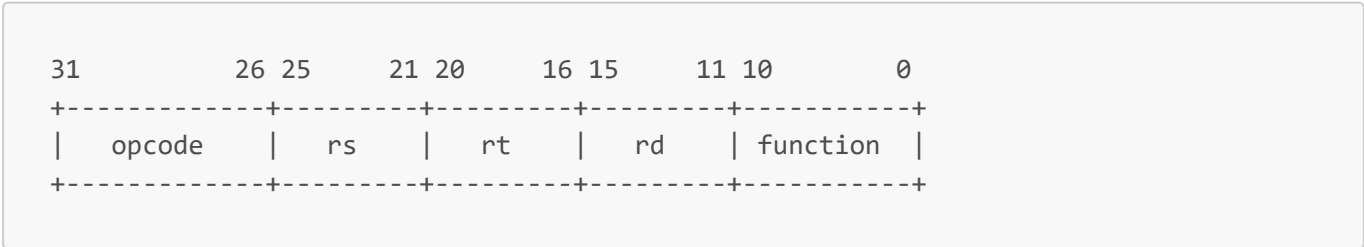
Instruction format defines how bits are arranged inside a machine instruction.

Common fields:

Field	Meaning
Opcode	Operation to perform
Source register	Input register
Destination register	Output register
Immediate value	Constant value
Address field	Memory or branch target information
Function field	Extra operation details

Example Format

Simple 32-bit instruction:



This resembles a RISC-style register instruction.

Types of Instruction Formats

Format	Used For	Example
Register format	Register-register operations	<code>ADD R1, R2, R3</code>

Format	Used For	Example
Immediate format	Constant values	ADDI R1, R2, 5
Memory format	Load/store	LOAD R1, [R2 + 8]
Branch format	Control flow	BEQ R1, R2, label

Why It Matters

Instruction format affects:

- Decoder complexity
- Instruction size
- Program size
- Pipeline design
- Hardware cost

Common Interview Angle

Question: "Why do many RISC architectures use fixed-length instructions?"

Answer: "Fixed-length instructions simplify fetching, decoding, and pipelining because the CPU can predict instruction boundaries easily."

3.11 Machine Instructions

What It Means

Machine instructions are binary instructions directly understood by the CPU.

Assembly language is a human-readable representation of machine instructions.

Example:

```
Assembly: ADD R1, R2, R3
Machine:  000000 00010 00011 00001 00000 100000
```

Main Categories

Category	Purpose	Examples
Data transfer	Move data	LOAD, STORE, MOV
Arithmetic	Numeric operations	ADD, SUB, MUL
Logical	Bit operations	AND, OR, XOR
Control flow	Change PC	JMP, BEQ, CALL, RET
Comparison	Set flags or compare	CMP, TEST

Category	Purpose	Examples
System	OS/hardware interaction	<code>SYSCALL</code> , <code>INT</code> , <code>HLT</code>

Why It Matters

Machine instructions are the final form of executable code. Understanding them helps with debugging, reverse engineering, compiler basics, and performance tuning.

Example

C loop:

```
for (int i = 0; i < 10; i++) {  
    sum += i;  
}
```

Machine-level idea:

```
MOV R1, 0      ; i = 0  
MOV R2, 0      ; sum = 0  
LOOP:  
ADD R2, R2, R1 ; sum += i  
ADD R1, R1, 1  ; i++  
CMP R1, 10  
BLT LOOP
```

Common Interview Angle

Question: "Difference between assembly and machine code?"

Answer: "Machine code is binary instructions executed by the CPU. Assembly is the symbolic human-readable form of those instructions."

3.12 Microprogrammed vs Hardwired Control

What It Means

The control unit generates signals that coordinate CPU operations. It can be implemented using hardwired logic or microprogramming.

Hardwired Control

Hardwired control uses fixed digital logic circuits to generate control signals.

Features:

- Faster
- Less flexible
- Harder to modify
- Common in simpler RISC designs

Microprogrammed Control

Microprogrammed control stores control steps as microinstructions in control memory.

Features:

- Easier to modify
- Good for complex instructions
- Usually slower than hardwired control
- Common historically in CISC designs

Example

For a complex instruction like:

```
ADD [A], [B]
```

The CPU may internally perform microinstructions:

```
MAR <- A
MDR <- Memory[MAR]
TEMP <- MDR
MAR <- B
MDR <- Memory[MAR]
TEMP <- TEMP + MDR
Memory[A] <- TEMP
```

Why It Matters

This topic connects ISA design to control unit implementation.

Common Interview Angle

Question: "Which is faster: hardwired or microprogrammed control?"

Answer: "Hardwired control is generally faster because control signals are generated directly by logic circuits. Microprogrammed control is more flexible but usually slower due to microinstruction sequencing."

3.13 Micro-Operations

What It Means

Micro-operations are the small internal operations performed by the CPU to execute one machine instruction.

A machine instruction may break into several micro-operations.

Examples

Fetch phase:

```
MAR <- PC
MDR <- Memory[MAR]
IR  <- MDR
PC  <- PC + 4
```

Execute **ADD R1, R2, R3**:

```
A <- R2
B <- R3
R1 <- A + B
```

Types of Micro-Operations

Type	Example
Register transfer	R1 <- R2
Arithmetic	R1 <- R2 + R3
Logic	R1 <- R2 AND R3
Shift	R1 <- R1 << 1
Memory	MDR <- Memory[MAR]

Why It Matters

Micro-operations explain how complex instructions are implemented internally and how datapath/control signals work.

Common Interview Angle

Question: "Is a micro-operation the same as a machine instruction?"

Answer: "No. A machine instruction is visible to software. Micro-operations are internal hardware-level steps used to implement that instruction."

3.14 Datapath and Control Path

What It Means

The datapath is the part of the CPU where data flows and gets processed. The control path generates signals that direct the datapath.

Part	Role
Datapath	Carries and transforms data
Control path	Tells datapath what to do

Datapath Includes

- Registers
- ALU
- Multiplexers
- Buses
- Memory interface
- Sign extension units
- Shifters

Control Path Includes

- Instruction decoder
- Control signal generator
- Sequencer
- Branch decision logic
- Pipeline control logic

Mental Model

```

Instruction bits
  |
  v
+-----+
| Control Path |
| Decode signals |
+-----+
      |
      | control signals
      v
+-----+
| Datapath |
| Registers -> ALU -> Registers |
+-----+

```

Why It Matters

Most CPU design questions become easier when you separate:

- "Where does the data move?" - datapath

- "Who tells it to move?" - control path

Common Interview Angle

Question: "Difference between datapath and control path?"

Answer: "Datapath stores and processes data using registers, ALU, buses, and memory interface. Control path decodes instructions and generates signals to control the datapath."

3.15 Superscalar Processors

What It Means

A superscalar processor can issue and execute multiple instructions in the same clock cycle using multiple execution units.

Instead of:

```
Cycle 1: Execute instruction 1
Cycle 2: Execute instruction 2
Cycle 3: Execute instruction 3
```

It may do:

```
Cycle 1: Execute instruction 1 and instruction 2
Cycle 2: Execute instruction 3 and instruction 4
```

Why It Matters

Superscalar execution improves instruction-level parallelism without requiring the programmer to explicitly write parallel code.

Requirements

The CPU must detect whether instructions are independent.

Example:

```
ADD R1, R2, R3
SUB R4, R5, R6
```

These can execute in parallel because they use different registers.

But:

```
ADD R1, R2, R3
SUB R4, R1, R6
```

The second instruction depends on **R1**, so it must wait for the first.

Common Interview Angle

Question: "What is a superscalar processor?"

Answer: "A superscalar processor can issue multiple instructions per cycle to multiple execution units, as long as dependencies and resource constraints allow."

3.16 VLIW Architectures

What It Means

VLIW stands for Very Long Instruction Word. A VLIW instruction contains multiple operations packed together, intended to execute in parallel.

Core Idea

In superscalar processors, hardware finds parallelism dynamically. In VLIW processors, the compiler finds parallelism statically and packs independent operations into one long instruction.

Example:

```
VLIW Instruction:
+-----+-----+-----+-----+
| ALU op  | Load op | Store op | Branch op |
+-----+-----+-----+-----+
```

Why It Matters

VLIW reduces hardware complexity but depends heavily on compiler quality.

Example

Independent operations:

```
ADD R1, R2, R3
LOAD R4, [R5]
MUL R6, R7, R8
```

A VLIW compiler may pack them into one wide instruction.

Common Interview Angle

Question: "Difference between superscalar and VLIW?"

Answer: "Superscalar processors use hardware to find and issue independent instructions at runtime. VLIW relies on the compiler to schedule independent operations into long instruction words before execution."

3.17 Out-of-Order Execution Internals

What It Means

Out-of-order execution allows a CPU to execute later independent instructions before earlier stalled instructions, while still preserving correct program results.

Why It Matters

Memory access, cache misses, and long-latency operations can stall a CPU. Out-of-order execution keeps execution units busy by running independent work while waiting.

Simple Example

```
1. LOAD R1, [memory]    ; may take many cycles
2. ADD  R2, R3, R4       ; independent
3. SUB  R5, R1, R6       ; depends on instruction 1
```

If the load stalls, instruction 2 can execute before instruction 1 completes. Instruction 3 must wait because it needs **R1**.

Main Internal Structures

Structure	Purpose
Instruction queue	Holds decoded instructions waiting to execute
Register renaming	Removes false dependencies
Reservation stations	Hold instructions until operands are ready
Reorder buffer	Commits results in original program order
Load/store queue	Handles memory ordering
Execution units	ALU, FPU, load/store units, branch units
Branch predictor	Predicts control flow to keep pipeline full

Register Renaming

Register renaming maps architectural registers to physical registers.

It removes false dependencies such as:

```
1. ADD R1, R2, R3
2. MUL R1, R4, R5
```

Both write to **R1**, but the second does not need the first result. The CPU can map them to different physical registers internally.

Reorder Buffer

The reorder buffer ensures that even if instructions execute out of order, they retire in original program order.

This preserves precise exceptions and correct program behavior.

Step-by-Step Flow

```
Fetch -> Decode -> Rename -> Dispatch -> Issue -> Execute -> Write result ->
Commit
```

Stage	Meaning
Fetch	Bring instructions into CPU
Decode	Convert instruction bits into internal operations
Rename	Map architectural registers to physical registers
Dispatch	Place operations into scheduling structures
Issue	Send ready operations to execution units
Execute	Perform operation
Write result	Broadcast result to waiting operations
Commit	Retire instruction in program order

Common Interview Angle

Question: "How can out-of-order execution still produce correct results?"

Answer: "The CPU tracks dependencies, uses register renaming to avoid false dependencies, executes only ready instructions, and commits results in program order using a reorder buffer."

4. Real-World Example

Example: What Happens When a Backend Server Handles a Request

Suppose a backend server receives an HTTP request and executes this code:


```
int total = price * quantity;
if (total > 1000) {
    total = total - discount;
}
return total;
```

At the processor level:

1. The program's compiled machine instructions are stored in memory.
2. The program counter points to the next instruction.
3. CPU fetches an instruction into the instruction register.
4. Control unit decodes the instruction.
5. Registers hold `price`, `quantity`, `total`, and `discount`.
6. ALU or multiplier computes `price * quantity`.
7. A comparison instruction checks whether `total > 1000`.
8. A branch instruction may change the program counter.
9. Store/load instructions move data between registers, stack, heap, and cache.
10. Modern CPU may execute independent instructions in parallel using superscalar and out-of-order execution.

Why This Matters for SDE Interviews

If the server is slow, the cause may not be just "bad code." It may involve:

- Too many memory accesses
- Poor cache locality
- Branch mispredictions
- Expensive synchronization
- CPU pipeline stalls
- Inefficient generated machine code

Example:

```
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
```

This is CPU-friendly because array elements are contiguous in memory.

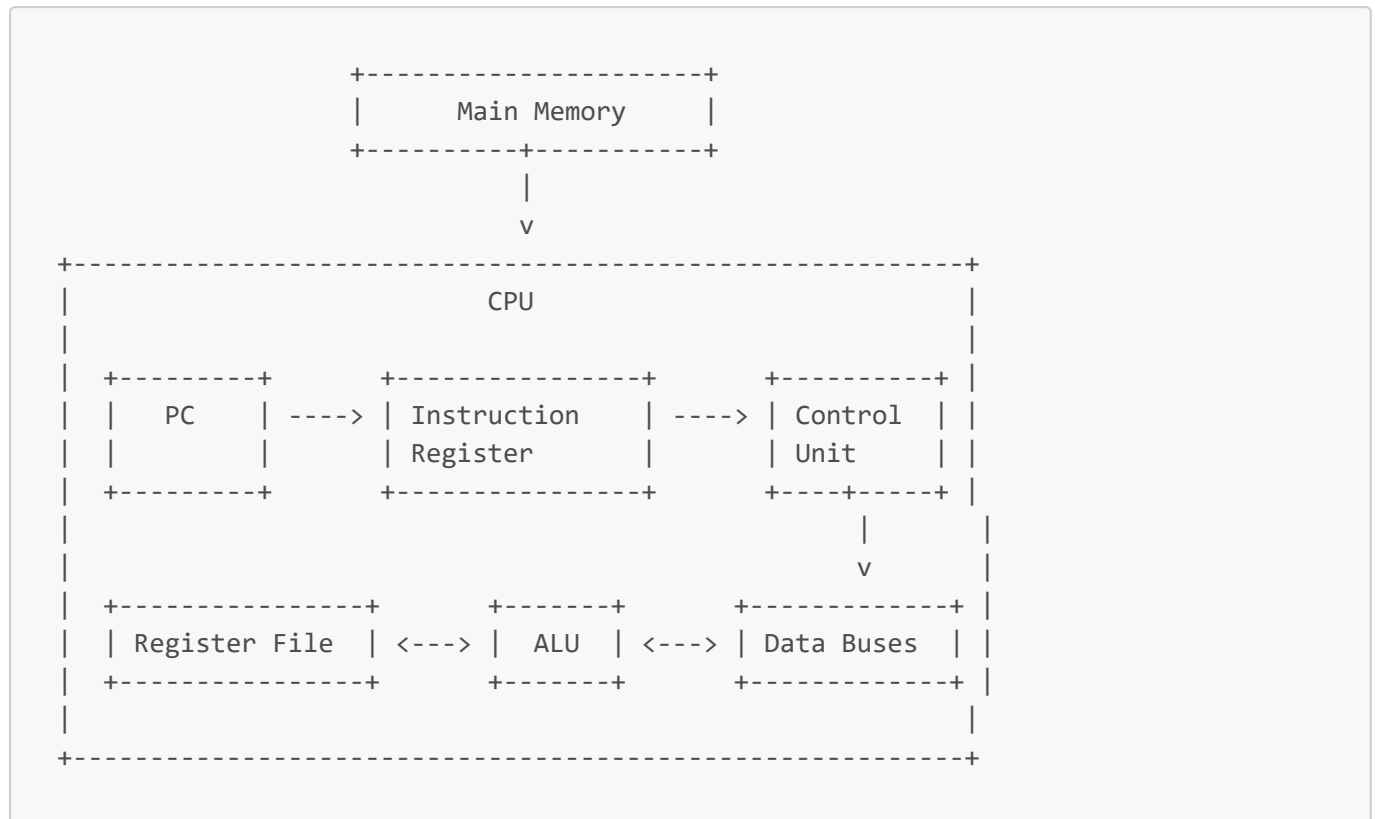
But:

```
for (Node* p = head; p != nullptr; p = p->next) {
    sum += p->value;
}
```

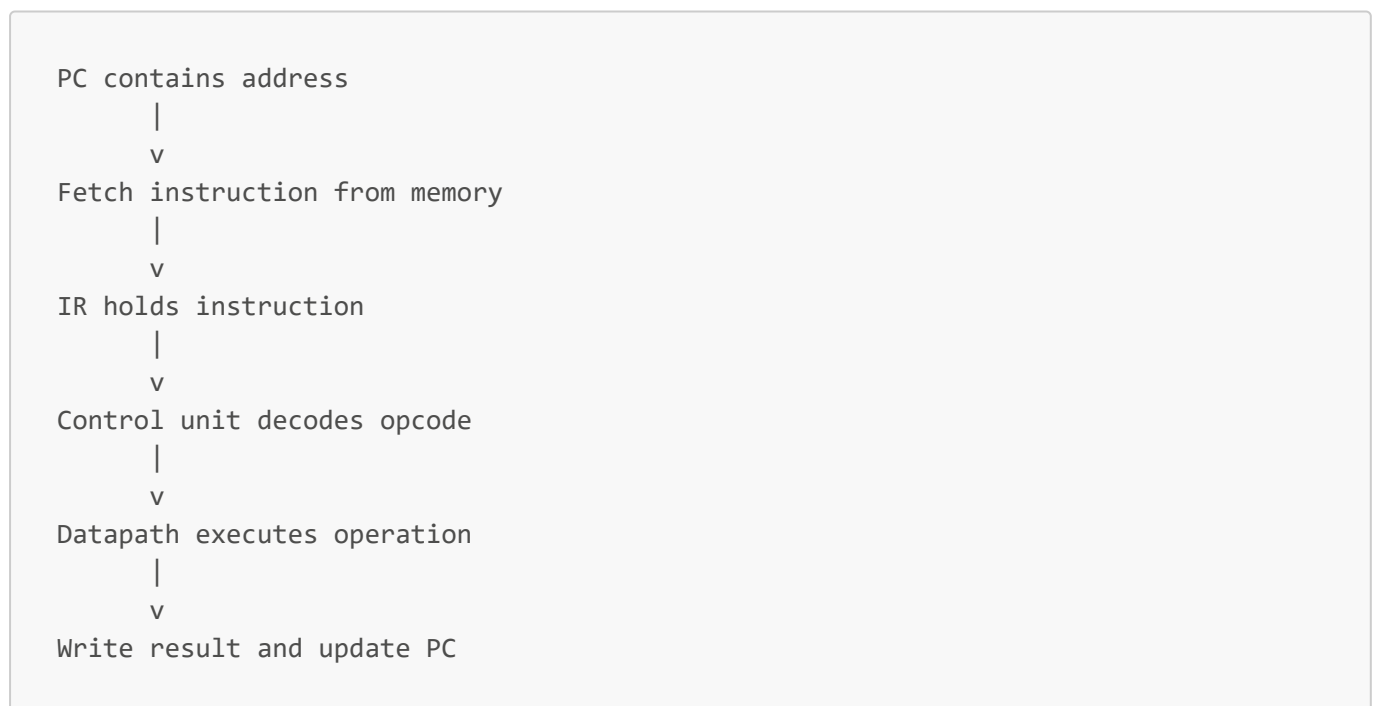
This can be slower because linked list nodes may be scattered in memory, causing cache misses and stalls.

5. Diagrams / Mental Models

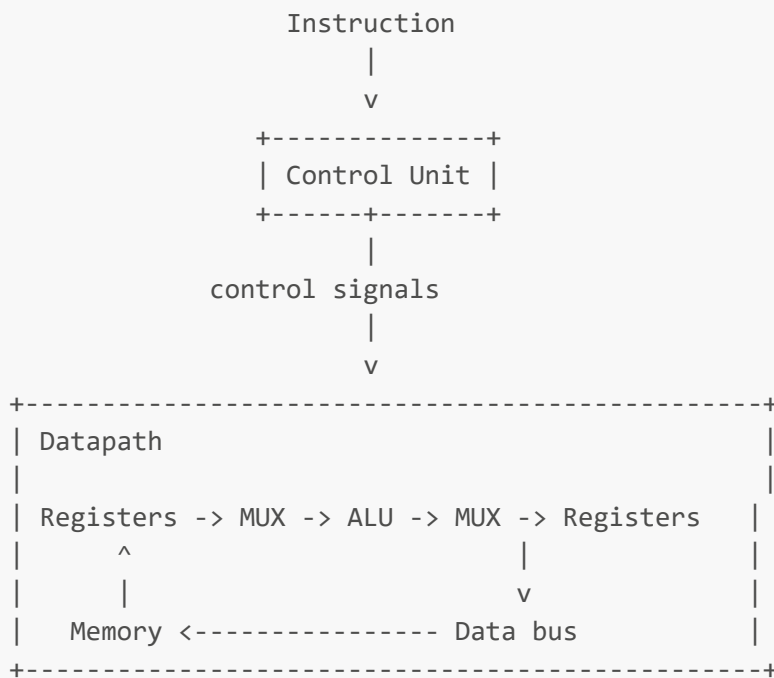
Basic CPU Organization



Fetch-Decode-Execute with PC and IR



Datapath vs Control Path



Out-of-Order Execution Mental Model

Program order:

I1 -> I2 -> I3 -> I4 -> I5

Execution order:

I1 starts

I2 waits for I1

I3 independent, executes early

I4 independent, executes early

I2 executes when data is ready

I5 executes

Commit order:

I1 -> I2 -> I3 -> I4 -> I5

Key idea: execution may be out of order, but final visible results are committed in order.

6. Common Interview Questions

1. What are the main components of a CPU?

Answer: The main CPU components are the ALU, control unit, registers, program counter, instruction register, datapath, and memory/bus interface. The ALU performs arithmetic and logical operations. The control unit decodes instructions and generates control signals. Registers hold temporary data and control information.

Interviewer expects:

- ALU computes

- Control unit coordinates
- Registers store fast temporary values
- PC and IR control instruction flow

Common mistakes:

- Forgetting program counter and instruction register
- Saying CPU only contains ALU and memory
- Confusing cache with register file

2. What is the function of the ALU?

Answer: The ALU performs arithmetic and logical operations such as addition, subtraction, bitwise AND, OR, XOR, shifts, and comparisons. It receives operands from registers and sends results back to registers or other datapath elements.

Interviewer expects:

- Arithmetic operations
- Logical operations
- Comparisons and flags

Common mistakes:

- Saying ALU stores instructions
- Ignoring logical operations
- Thinking ALU directly controls memory

3. What is the role of the control unit?

Answer: The control unit decodes the current instruction and generates control signals that direct the datapath. It decides which registers to read, which ALU operation to perform, whether memory should be read or written, and how the PC should be updated.

Interviewer expects:

- Instruction decoding
- Control signal generation
- Coordination of CPU units

Common mistakes:

- Saying control unit performs arithmetic
- Not mentioning control signals
- Confusing control unit with operating system

4. What is the program counter?

Answer: The program counter is a CPU register that stores the address of the next instruction to fetch. It usually increments after each instruction, but branches, jumps, calls, returns, and interrupts can load a different address into it.

Interviewer expects:

- Stores address, not instruction
- Controls next instruction
- Changes during branch/jump

Common mistakes:

- Saying PC stores data
- Saying PC stores current instruction bits
- Forgetting branches modify PC

5. What is the instruction register?

Answer: The instruction register stores the instruction currently fetched from memory. The control unit reads the instruction register to decode the opcode, operands, and addressing mode.

Interviewer expects:

- Holds current instruction
- Used during decode
- Different from PC

Common mistakes:

- Confusing IR with PC
- Saying IR stores address of next instruction
- Forgetting it is inside CPU

6. Explain the fetch-decode-execute cycle.

Answer: In the fetch stage, the CPU uses the PC to read the next instruction from memory into the IR. In decode, the control unit identifies the opcode, operands, and addressing mode. In execute, the datapath performs the operation using ALU, registers, or memory. Finally, results are written back and the PC is updated.

Interviewer expects:

- Fetch from PC
- Decode in control unit
- Execute through datapath
- PC update

Common mistakes:

- Skipping PC update
- Ignoring memory access and write-back
- Treating decode and execute as the same thing

7. What is an Instruction Set Architecture?

Answer: An ISA is the programmer-visible specification of a processor. It defines instructions, registers, addressing modes, data types, memory behavior, exceptions, and privilege levels. It is the interface between software and hardware.

Interviewer expects:

- Interface between hardware and software
- Defines instructions and registers
- Different from microarchitecture

Common mistakes:

- Saying ISA is the physical CPU design
- Confusing ISA with assembly language only
- Not mentioning programmer-visible behavior

8. What is the difference between RISC and CISC?

Answer: RISC uses simpler, usually fixed-length instructions and typically follows load-store architecture. CISC uses more complex, often variable-length instructions that may directly operate on memory. RISC is easier to pipeline, while CISC can have denser code and supports complex legacy instructions.

Interviewer expects:

- RISC: simple, fixed-length, load-store
- CISC: complex, variable-length, many addressing modes
- Modern CPUs blur the distinction

Common mistakes:

- Saying RISC means fewer instructions only
- Saying CISC is always slower
- Ignoring modern internal micro-operations

9. What are addressing modes?

Answer: Addressing modes specify how an instruction locates its operands. Examples include immediate, register, direct, indirect, base plus offset, indexed, relative, and implied addressing.

Interviewer expects:

- Method of finding operands
- Examples
- Relation to arrays, pointers, stack, and branches

Common mistakes:

- Confusing addressing modes with memory hierarchy
- Listing examples without explaining operand location
- Forgetting immediate and relative addressing

10. What is the difference between hardwired and microprogrammed control?

Answer: Hardwired control uses fixed logic circuits to generate control signals, making it fast but less flexible. Microprogrammed control stores control steps as microinstructions in control memory, making it easier to modify and suitable for complex instructions but usually slower.

Interviewer expects:

- Hardwired: fast, fixed logic
- Microprogrammed: flexible, control memory
- CISC often benefits from microprogramming

Common mistakes:

- Saying microprogramming is software run by OS
- Saying hardwired control is always better
- Not mentioning control signals

11. What are micro-operations?

Answer: Micro-operations are small internal CPU operations used to implement machine instructions. Examples include register transfer, memory read, ALU operation, and PC increment.

Interviewer expects:

- Internal low-level steps
- Not directly visible to normal programs
- Used in instruction execution

Common mistakes:

- Treating micro-operations as assembly instructions
- Forgetting register transfer operations
- Ignoring fetch-stage micro-operations

12. What is the difference between datapath and control path?

Answer: The datapath contains hardware that stores, moves, and processes data, such as registers, ALU, buses, and multiplexers. The control path decodes instructions and sends signals that tell the datapath what to do.

Interviewer expects:

- Datapath handles data
- Control path handles decisions/signals
- Both cooperate during instruction execution

Common mistakes:

- Saying datapath and control path are the same
- Not mentioning control signals
- Ignoring multiplexers and buses

13. What is a superscalar processor?

Answer: A superscalar processor can issue multiple instructions per clock cycle to multiple execution units, provided the instructions are independent and resources are available.

Interviewer expects:

- Multiple issue per cycle
- Multiple execution units
- Dependency checking

Common mistakes:

- Confusing superscalar with multicore
- Thinking it always executes every instruction in parallel
- Ignoring data hazards

14. What is VLIW?

Answer: VLIW, or Very Long Instruction Word, is an architecture where one long instruction contains multiple independent operations that execute in parallel. The compiler schedules these operations before execution.

Interviewer expects:

- Compiler-managed parallelism
- Long instruction word
- Less dynamic hardware scheduling

Common mistakes:

- Saying VLIW dynamically schedules like superscalar
- Ignoring compiler responsibility
- Assuming it is same as SIMD

15. What is out-of-order execution?

Answer: Out-of-order execution lets the CPU execute ready independent instructions before earlier stalled instructions. The CPU tracks dependencies and commits results in program order to preserve correct behavior.

Interviewer expects:

- Executes independent instructions early
- Handles dependencies
- Uses structures like reorder buffer and register renaming

Common mistakes:

- Saying program output changes
- Forgetting in-order commit
- Ignoring false dependencies

7. Deep-Dive Questions

1. How does register renaming help out-of-order execution?

Answer: Register renaming maps architectural registers visible to the programmer onto a larger set of physical registers inside the CPU. This removes false dependencies such as write-after-write and write-after-read hazards.

Example:

```
1. ADD R1, R2, R3
2. MUL R1, R4, R5
```

Both instructions write to architectural **R1**, but they are independent if no later instruction needs the first **R1**. Register renaming assigns separate physical registers, allowing both operations to proceed without unnecessary waiting.

2. Why is a reorder buffer needed in out-of-order processors?

Answer: A reorder buffer tracks instructions after they are issued and allows them to commit in original program order. This ensures precise exceptions and correct architectural state. If an exception or branch misprediction occurs, the CPU can discard speculative work after the problematic instruction.

Key point: Out-of-order execution changes internal execution order, not the final visible program order.

3. Why does branch prediction matter in modern processors?

Answer: Modern processors have deep pipelines and fetch many instructions ahead. When a branch appears, the CPU must guess which path will execute next. Good branch prediction keeps the pipeline full. A wrong prediction causes speculative instructions to be discarded, wasting cycles.

Example:

```
if (x > 0) {
    y++;
}
```

The CPU may predict whether the branch is taken before the condition is fully resolved.

4. Why can memory operations make out-of-order execution difficult?

Answer: Memory dependencies are harder to detect than register dependencies because two memory addresses may or may not refer to the same location. The CPU uses load/store queues and memory disambiguation to decide whether a load can safely execute before an earlier store.

Example:

```
STORE [R1], R2
LOAD  R3, [R4]
```

If **R1** and **R4** point to the same address, reordering could be incorrect. If they are different, reordering may be safe.

5. How do CISC processors execute complex instructions efficiently?

Answer: Modern CISC processors, especially x86, often decode complex instructions into simpler internal micro-operations. These micro-operations are then scheduled and executed by a pipeline that may look internally similar to a RISC-like engine.

This allows the CPU to preserve backward compatibility with complex ISA instructions while still using modern high-performance execution techniques.

8. Comparison Tables

RISC vs CISC

Feature	RISC	CISC
Instruction complexity	Simple	Complex
Instruction length	Usually fixed	Often variable
Memory access	Load/store only	Many instructions can access memory
Addressing modes	Fewer	More
Decoding	Simpler	More complex
Compiler role	More important	Hardware handles more complexity
Pipelining	Easier	Harder, but modern CPUs handle it
Code size	May be larger	Often compact
Examples	ARM, RISC-V, MIPS	x86/x86-64

Hardwired vs Microprogrammed Control

Feature	Hardwired Control	Microprogrammed Control
Implementation	Logic circuits	Control memory with microinstructions
Speed	Faster	Usually slower
Flexibility	Low	High
Modification	Difficult	Easier
Best suited for	Simple, regular instruction sets	Complex instruction sets
Common association	RISC	CISC

Superscalar vs VLIW

Feature	Superscalar	VLIW
Parallelism discovered by	Hardware at runtime	Compiler before execution
Instruction format	Normal instructions	Very long instruction word
Hardware complexity	Higher	Lower
Compiler complexity	Moderate	High
Handles dynamic stalls	Better	Harder
Binary compatibility	Easier across implementations	Harder if hardware width changes
Example idea	CPU issues multiple ready instructions	Compiler packs multiple ops together

In-Order vs Out-of-Order Execution

Feature	In-Order Execution	Out-of-Order Execution
Execution order	Program order	Ready instructions may execute early
Hardware complexity	Lower	Higher
Performance	Lower on stalls	Higher for independent work
Dependency tracking	Simpler	Advanced
Structures needed	Basic pipeline control	Rename table, ROB, reservation stations
Power usage	Lower	Higher
Used in	Simple embedded CPUs	High-performance desktop/server CPUs

Register Architecture Comparison

Type	Main Idea	Pros	Cons
Stack	Operands on stack	Compact instructions	More memory/stack traffic
Accumulator	One implicit register	Simple hardware	Limited parallelism
Register-memory	ALU can use memory operands	Flexible, compact code	More complex decoding
Load-store	ALU uses registers only	Simple pipeline	More instructions for memory data

PC vs IR

Register	Full Form	Stores	Used For
PC	Program counter	Address of next instruction	Instruction fetch and control flow

Register	Full Form	Stores	Used For
IR	Instruction register	Current instruction	Decode and execution control

9. Common Mistakes

- Thinking the CPU directly executes C/C++/Java code. It executes machine instructions.
- Confusing ISA with microarchitecture. ISA is the contract; microarchitecture is the implementation.
- Saying the program counter stores the current instruction. It stores an address.
- Saying the instruction register stores the next address. It stores the fetched instruction.
- Assuming RISC always means faster and CISC always means slower.
- Forgetting that modern x86 CPUs often break instructions into micro-operations.
- Confusing superscalar processors with multicore processors.
- Thinking out-of-order execution changes the final result order. It commits in program order.
- Ignoring memory access cost when discussing CPU performance.
- Treating registers, cache, and RAM as the same thing.
- Forgetting that branch instructions change the program counter.
- Assuming VLIW and superscalar are the same because both execute multiple operations.
- Saying hardwired control cannot implement complex instructions at all. It can, but complexity grows.
- Saying microprogramming means normal application software. It is internal CPU control logic.

10. Edge Cases / Special Cases

Branches and Program Counter Updates

Most instructions increment the PC normally. Branches, jumps, calls, returns, exceptions, and interrupts can load a different value into the PC.

Variable-Length Instructions

In CISC architectures like x86, instructions may have different lengths. This makes fetching and decoding more complex than fixed-length RISC instructions.

Precise Exceptions

Out-of-order CPUs may execute instructions out of order internally, but exceptions must appear as if instructions executed in program order. The reorder buffer helps achieve this.

Memory Aliasing

Two different address expressions may refer to the same memory location.

Example:

```
*p = 10;
x = *q;
```

If `p` and `q` point to the same address, reordering these operations may be unsafe.

Self-Modifying Code

If code modifies instructions in memory, the CPU must ensure instruction cache and pipeline state remain consistent. Modern systems discourage this except in controlled cases like JIT compilers.

Speculative Execution

Modern CPUs may execute instructions before knowing for sure whether they are needed. If the prediction is wrong, results are discarded. However, side effects in caches can still matter for security.

Flags Register

Some architectures use condition flags such as zero, carry, sign, and overflow. Other architectures prefer explicit compare results or condition registers.

Load-Use Hazard

An instruction that immediately uses the result of a load may stall if memory data is not ready.

```
LOAD R1, [x]
ADD  R2, R1, R3
```

Interrupts

An interrupt can pause normal execution, save current CPU state, jump to an interrupt handler, then resume later.

Micro-Operation Fusion

Some modern CPUs can combine certain micro-operations internally to reduce pipeline work. This is implementation-specific and not part of the ISA.

11. How to Explain in Interview

Processor architecture describes how a CPU is organized and how it executes instructions. A program is stored as machine instructions in memory. The program counter gives the address of the next instruction, the instruction register holds the fetched instruction, the control unit decodes it, and the datapath executes it using registers and the ALU. The ISA defines what instructions software can use, while the microarchitecture defines how a particular CPU implements them. Modern CPUs improve performance using pipelining, superscalar issue, branch prediction, register renaming, and out-of-order execution while preserving correct program behavior.

12. Quick Revision Notes

Key Definitions

Term	Definition
CPU	Hardware unit that executes machine instructions

Term	Definition
ALU	Performs arithmetic and logic
Control unit	Decodes instructions and generates control signals
Register	Very fast storage inside CPU
Program counter	Address of next instruction
Instruction register	Current fetched instruction
ISA	Software-visible instruction set contract
Microarchitecture	Hardware implementation of an ISA
Addressing mode	Method of locating operands
Micro-operation	Internal step used to execute an instruction
Datapath	Hardware where data moves and gets processed
Control path	Hardware that controls datapath actions

Important Points

- CPU execution is based on fetch, decode, execute, write-back, and PC update.
- Registers are faster than memory because they are inside the CPU.
- ISA is not the same as physical CPU design.
- RISC favors simple instructions and load-store design.
- CISC supports more complex instructions and addressing modes.
- Hardwired control is faster; microprogrammed control is more flexible.
- Superscalar CPUs issue multiple instructions per cycle.
- VLIW relies on compiler-scheduled parallelism.
- Out-of-order CPUs execute ready instructions early but commit in order.

Common Comparisons

Comparison	Must Remember
PC vs IR	PC stores address; IR stores instruction
RISC vs CISC	Simple fixed instructions vs complex variable instructions
Hardwired vs Microprogrammed	Fast fixed logic vs flexible control memory
Superscalar vs VLIW	Hardware scheduling vs compiler scheduling
In-order vs Out-of-order	Program-order execution vs dynamic ready-first execution
ISA vs Microarchitecture	Contract vs implementation

Must-Remember Facts

- Every high-level program becomes machine instructions.

- The control unit does not perform arithmetic; the ALU does.
- The PC changes during branches, calls, returns, interrupts, and exceptions.
- Addressing modes are about finding operands.
- Out-of-order execution needs dependency tracking and in-order commit.
- Superscalar is not the same as multicore.
- Modern CISC processors may internally use RISC-like micro-operations.

Interview Traps

- Do not say "RISC has fewer instructions" as the only difference.
- Do not say "out-of-order means final result is out of order."
- Do not confuse assembly with ISA.
- Do not ignore memory hierarchy when discussing CPU performance.
- Do not call microprogramming normal software programming.

13. Practice Tasks

Task 1: Trace a Simple Instruction Sequence

Trace PC, IR, registers, and memory for:

```
1000: LOAD R1, [2000]
1004: LOAD R2, [2004]
1008: ADD  R3, R1, R2
1012: STORE [2008], R3
```

Assume:

```
Memory[2000] = 7
Memory[2004] = 5
```

Expected result:

```
R1 = 7
R2 = 5
R3 = 12
Memory[2008] = 12
```

Task 2: Identify Addressing Modes

For each instruction, identify the addressing mode:

```
MOV R1, #10
ADD R2, R3
```

```
LOAD R4, [1000]
LOAD R5, [R6]
LOAD R7, [R8 + 12]
BEQ R1, R2, label
```

Expected:

Instruction	Addressing Mode
MOV R1, #10	Immediate
ADD R2, R3	Register
LOAD R4, [1000]	Direct
LOAD R5, [R6]	Indirect
LOAD R7, [R8 + 12]	Base + offset
BEQ R1, R2, label	Relative/branch addressing

Task 3: Convert High-Level Code to Assembly-Like Steps

Convert:

```
z = (x + y) * 2;
```

Possible answer:

```
LOAD R1, [x]
LOAD R2, [y]
ADD R3, R1, R2
MUL R4, R3, 2
STORE [z], R4
```

Task 4: Draw the Fetch-Decode-Execute Cycle

Draw a diagram showing:

- PC
- Memory
- Instruction register
- Control unit
- ALU
- Registers
- PC update

Then explain it in five sentences.

Task 5: Compare RISC and CISC Using an Example

Take this operation:

```
memory[A] = memory[A] + 5
```

Write:

- One CISC-style instruction
- A RISC-style instruction sequence

Expected idea:

```
CISC:
ADD [A], #5

RISC:
LOAD R1, [A]
ADD R1, R1, #5
STORE [A], R1
```

Task 6: Spot Dependencies

For this sequence:

```
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
I3: MUL R6, R7, R8
I4: ADD R1, R9, R10
```

Identify:

- True dependency
- Independent instruction
- False dependency

Expected:

- I2 has a true dependency on I1 because it reads R1.
- I3 is independent.
- I4 has a write-after-write false dependency on R1, removable by register renaming.

Task 7: Explain Out-of-Order Execution

Use this sequence:

```
LOAD R1, [x]
ADD  R2, R3, R4
SUB  R5, R1, R6
```

Explain why `ADD R2, R3, R4` can execute before the `LOAD` completes, but `SUB R5, R1, R6` cannot.

Task 8: Design a Tiny Instruction Format

Design a 16-bit instruction format with:

- 4-bit opcode
- 4-bit destination register
- 4-bit source register 1
- 4-bit source register 2 or immediate field

Then encode a fake `ADD R1, R2, R3`.

Task 9: Explain CPU Performance Bottlenecks

Compare performance of:

```
vector<int> a;
list<int> b;
```

when summing all elements. Explain using memory locality, cache, and CPU stalls.

Task 10: Interview Simulation

Answer aloud in under two minutes:

"Explain how a CPU executes an instruction and how modern CPUs improve performance."

Your answer should include:

- PC
- IR
- Control unit
- ALU/registers
- ISA
- Pipeline/superscalar/out-of-order idea

14. Final Cheat Sheet

Core Definition

Processor architecture is the design and behavior of a CPU, including its ISA, registers, ALU, control unit, datapath, memory access, and execution techniques.

Why It Matters

It explains how software becomes hardware actions and why CPU-level details affect program performance, compiler behavior, operating systems, and real-world application speed.

Most Asked Questions

Question	One-Line Answer
What are CPU components?	ALU, control unit, registers, PC, IR, datapath, memory interface
What does ALU do?	Arithmetic, logic, shifts, comparisons
What does control unit do?	Decodes instructions and generates control signals
What is PC?	Register holding address of next instruction
What is IR?	Register holding current fetched instruction
What is ISA?	Software-visible contract of instructions and registers
RISC vs CISC?	Simple load-store instructions vs complex flexible instructions
What are addressing modes?	Ways instructions locate operands
Hardwired vs microprogrammed?	Fixed fast logic vs flexible control memory
What is superscalar?	Multiple instructions issued per cycle
What is VLIW?	Compiler-packed parallel operations in long instructions
What is out-of-order execution?	CPU executes ready independent instructions early and commits in order

Common Comparisons

Pair	Key Difference
PC vs IR	Address of next instruction vs current instruction
ISA vs Microarchitecture	Contract vs implementation
RISC vs CISC	Simple fixed load-store vs complex variable instructions
Superscalar vs VLIW	Hardware scheduling vs compiler scheduling
Hardwired vs Microprogrammed	Logic circuits vs microinstruction control memory
In-order vs Out-of-order	Sequential execution vs dependency-based dynamic execution

One-Line Interview Answer

Processor architecture defines how a CPU understands and executes machine instructions: the PC fetches instructions, the IR holds them, the control unit decodes them, the datapath uses registers and the ALU to

execute them, and modern CPUs improve performance with techniques like pipelining, superscalar issue, branch prediction, and out-of-order execution.